BiteBound

Simon Völkl, Johannes Schieder, and Ulrich Schäfer

***Abstract*—Designing responsive, cross-device IoT games on resource-constrained microcontrollers requires balancing low-latency local execution with secure, asynchronous network communication. This project presents BiteBound, a physical, tilt-controlled labyrinth game built on a Waveshare ESP32-S3-Touch-LCD-1.69 featuring real-time communication with two remote dashboards. The system utilizes a dual-core ESP32-S3 microcontroller to handle network communication, sensor inputs, physics simulation, game logic and display rendering, while a Kotlin-based Android app and a Node-RED interface provide bidirectional control via telemetry and command messages over the Message Queuing Telemetry Transport (MQTT) protocol. The firmware, running on a network and game logic core, achieves a stable 50 Hz frame rate. A bakery-themed frontend provides a playful user experience, while the companion interfaces successfully allow for remote monitoring and control of the game state during gameplay. BiteBound demonstrates a robust, responsive end-to-end architecture bridging physical embedded interactions with digital companion interfaces.**

***Index Terms*—Android, Arduino, ESP32-S3, FreeRTOS, IMU, IoT Game, MQTT, Node-RED**

I. Introduction

EMBEDDED Internet of Things (IoT) devices increasingly combine latency-sensitive local interaction with networked communication, forcing developers to reconcile deterministic real-time execution with asynchronous, potentially blocking network operations on limited hardware. Modern microcontrollers such as the ESP32-S3 address this tension by pairing dual-core processing with integrated wireless connectivity [4], enabling responsive interactive applications that remain observable and controllable over the network. Realizing such systems, however, requires careful task partitioning, thread-safe state sharing, and a lightweight publish-subscribe transport such as the Message Queuing Telemetry Transport (MQTT) protocol [2] to keep local and remote representations synchronized.

Tilt-controlled games are a compelling demonstrator for these constraints, as they couple a high-frequency sensor, physics, and rendering loop with continuous bidirectional data exchange. BiteBound instantiates this scenario on a Waveshare ESP32-S3-Touch-LCD-1.69 board, running a dual-core FreeRTOS firmware [6] that isolates the 50 Hz game loop from the MQTT network stack [1]. Two companion interfaces, a native Kotlin Android application [19] and a Node-RED dashboard [27], provide remote monitoring and control, together forming a complete end-to-end embedded IoT system.

The objective of the BiteBound project is to implement a highly responsive, low-latency gaming platform demonstrating efficient multi-core task scheduling on resource-constrained hardware. Centered on a playful, bakery-themed aesthetic, the system requires navigating a virtual cherry (ball) through procedural mazes. The main technical challenge is maintaining a stable, high-frame-rate rendering pipeline alongside bidirectional MQTT communication, supported by native Android and Node-RED dashboards to deliver a unified, multi-platform user experience.

The authors are with the Department of Electrical Engineering, Media, and Computer Science, Ostbayerische Technische Hochschule Amberg-Weiden, 92224 Amberg, Germany (e-mail: {s.voelkl2, j.schieder, u.schaefer}@oth-aw.de).

Fig. 1. BiteBound Project Logo in a bakery theme. Designed by Simon Völkl.

The remainder of this report is organized as follows: Section II outlines the high-level system architecture. Section III details the firmware, sensor integration, physics, game logic, and display optimizations. The companion interfaces are presented in Section IV (Android App) and Section V (Node-RED). Finally, Section VI covers project organization and milestones, while Sections VII and VIII provide evaluation, conclusions, and future outlook.

II. Technical Concept

The technical concept of BiteBound is designed to ensure real-time responsiveness, modularity, and thread safety. By leveraging the dual-core capabilities of the ESP32-S3, the system separates latency-sensitive gameplay and rendering tasks from time-intensive network operations.

A. High-Level Architecture

The architecture relies on an asynchronous, event-driven design. The ESP32-S3 runs two primary execution threads across its dual cores to balance the computational workload. Core 1 is designated as the Game Core, measuring the sensor data, executing the physics engine, handling the game logic, and updating the display. Core 0 acts as the Network Core, managing secure TLS connections, MQTT messaging, and transmitting telemetry, running significantly slower than the Game Core.

Communication between the embedded hardware and the external control interfaces is bridged through the centralized MQTT broker HiveMQ [1], [2]. External clients communicate bidirectionally with the device. They send game-related commands to a dedicated topic, while subscribing to a telemetry topic to monitor real-time variables.

The system architecture is depicted in Figure 2, illustrating the interactions between the ESP32-S3 and the frontend dashboards.

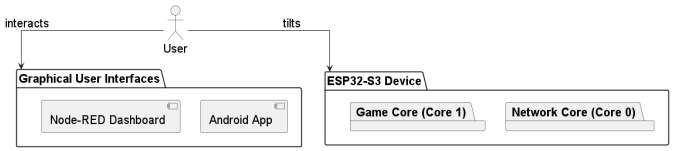

Fig. 2. System architecture diagram displaying the user interactions with the ESP32-S3 and the frontend dashboards.

B. Components

The system architecture is divided into three primary components:

- **Hardware (ESP32-S3):** The core of the system is the Waveshare ESP32-S3 development board equipped with a 1.69-inch touch LCD, a 6-axis inertial measurement unit (IMU), and a buzzer. It hosts the procedural sensor manager, maze generator, 2D physics simulation, game engine, display logic, and network communication stack.
- **Android Application:** A native mobile interface written in Kotlin. It receives telemetry data and sends control commands to the ESP32-S3 via MQTT.
- **Node-RED Dashboard:** A flow-based UI-Builder dashboard that mirrors the application logic of the Android app.

III. Hardware

The firmware of the BiteBound platform is written in C++ using the Arduino framework [3]. The code operates on a dual-core microcontroller to separate timing-sensitive game updates from network Input/Output (I/O) activities, as illustrated in Figure 3.

A. ESP32-S3 Microcontroller

The core processing unit is a Waveshare ESP32-S3 1.69-inch Touch LCD development board [4]. Onboard peripherals integrated on the board include a QMI8658 six-axis Inertial Measurement Unit (IMU), a Real-Time Clock (RTC), and an ST7789-driven 240×280 pixel Liquid Crystal Display (LCD). Dedicated hardware pins and interface buses, like Inter-Integrated Circuit (I2C) for sensors and Serial Peripheral Interface (SPI) for the display, are managed centrally via the `config.h` configuration file [5].

B. Concurrency

Execution is distributed across both processor cores using FreeRTOS tasks to maintain stable game loop execution rates [6]:

- Core 1 (Game Core): Runs the main application thread executing the game loop at approximately 50 Hz (20 ms loop budget). This core is responsible for sensor readings, physics steps, collision resolution, game engine updates, and display rendering. It is designed to be non-blocking and deterministic to ensure smooth gameplay.
- Core 0 (Network Core): Executes the background network task (`vNetworkTask`) at approximately 2 Hz. This core handles secure TLS handshakes, manages the MQTT client connection, and serializes and publishes telemetry data.

Inter-process communication (IPC) between cores utilizes two distinct mechanisms to ensure thread safety and avoid race conditions [7]:

- Mutex Synchronization: The global state structure (`SharedStateData`) is protected with a FreeRTOS semaphore. Access is managed through a C++ Resource Acquisition Is Initialization (RAII) helper class (`MutexLock`) to automate the lock release cycle and avoid deadlock states [8].
- Command Queue: Unidirectional commands from Core 0 are transferred to Core 1 using a thread-safe FreeRTOS queue (`commandQueue`) with a length of 10. This structure prevents network tasks from causing latency or stalling the rendering pipeline [6].

C. Sensor Data Acquisition

Onboard data collection is managed by the `SensorManager` class, which reads physical values from the QMI8658 IMU over I2C and samples battery voltage through an internal Analog-to-Digital Converter (ADC). The raw accelerometer (a_x, a_y, a_z) and gyroscope $(\omega_x, \omega_y, \omega_z)$ readings are structured inside the `SensorData` format.

To filter out high-frequency noise and mechanical vibrations, the raw tilt inputs are smoothed using an Exponential Moving Average (EMA) filter [9]:

$$S_t = \alpha \cdot Y_t + (1 - \alpha) \cdot S_{t-1} \tag{1}$$

where S_t is the filtered value, Y_t is the current sensor input, and α is the smoothing coefficient (`ema_alpha` in configuration). A deadzone threshold is then applied to the output; inputs falling below this range are registered as zero, preventing drifting movement when the controller is resting.

D. Physics Simulation & Game Logic

The two-dimensional physics simulation is implemented inside the `PhysicsEngine` class. It is parameterized via the `PhysicsParams` structure, enabling real-time adjustments. The ball is modeled as a `PhysicsBody` with position $\vec{x}$, velocity $\vec{v}$, and radius r.

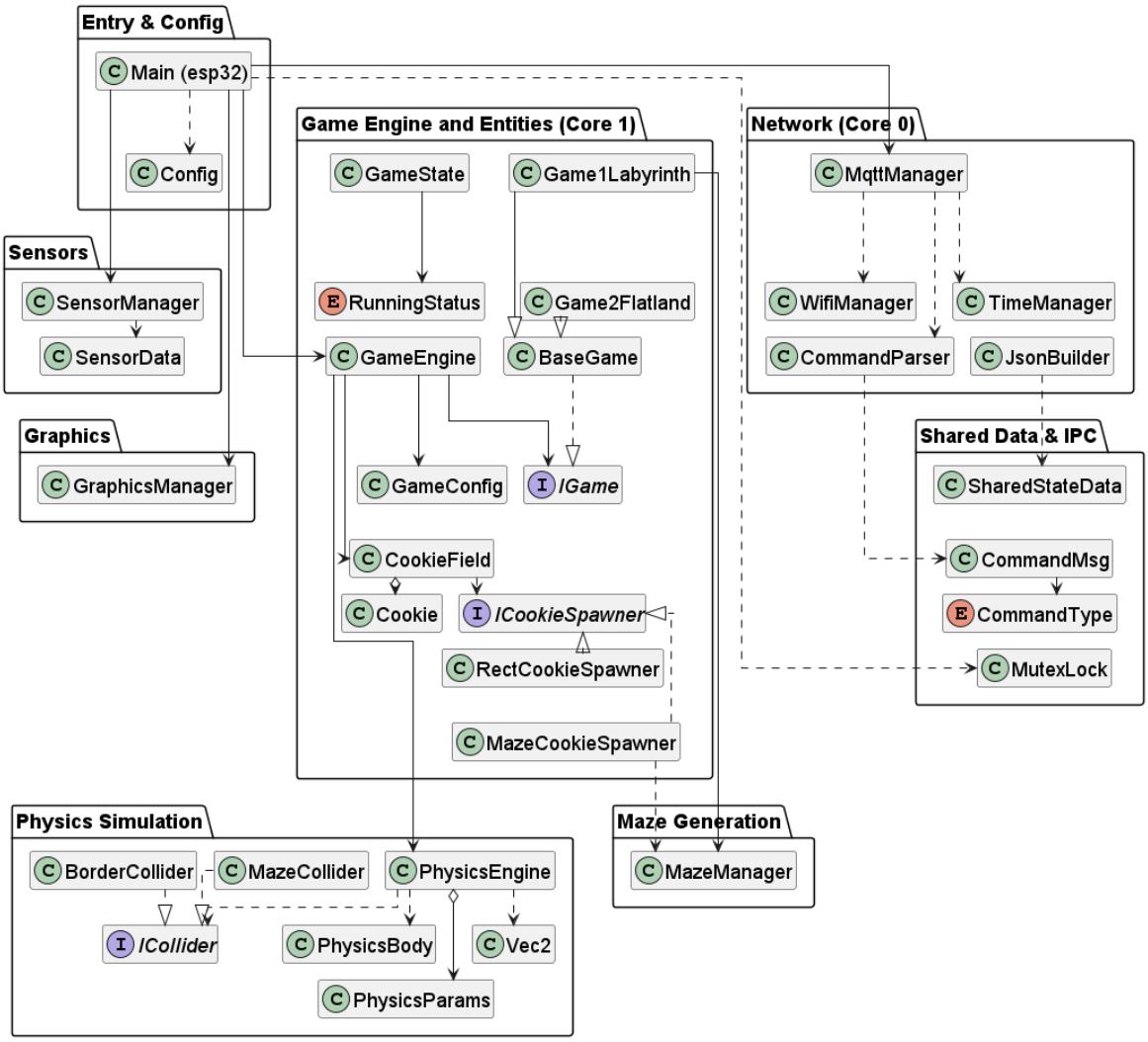

Fig. 3. UML class diagram of the BiteBound ESP32-S3 firmware architecture, showing the main components and their interactions.

The engine uses semi-implicit Euler integration [10] with a constant time step ($dt = 0.02\,\mathrm{s}$) to update kinematics:

$$\vec{v}_{t+1} = \vec{v}_t + \vec{a} \cdot dt \tag{2}$$

$$\vec{x}_{t+1} = \vec{x}_t + \vec{v}_{t+1} \cdot dt \tag{3}$$

where $\vec{a}$ is the scaled acceleration derived from the filtered tilt values. The velocity is capped at a configurable `maxSpeed` threshold. To prevent tunneling errors when dealing with fast movements or thin boundaries, sub-stepping is utilized during collision detection.

Collisions with the environment are resolved via the `ICollider` strategy interface. The `BorderCollider` class checks for rectangular screen boundary crossings, while the `MazeCollider` evaluates pixel-level collisions against the generated layout grid. Upon detecting a collision, the engine extracts the normal vector $\hat{n}$ and calculates the updated velocity according to a restitution model [11]:

$$\vec{v}_{\text{new}} = \vec{v} - (1 + e)(\vec{v} \cdot \hat{n})\hat{n} \tag{4}$$

where $e \in [0, 1]$ represents the bounce restitution factor. The tangential velocity remains unaffected, permitting the ball to slide along walls.

Collectibles are modeled using the `CookieField` class. When the physical body of the ball overlaps with an active cookie coordinate within a configured pickup radius, the score is incremented and the `ICookieSpawner` strategy interface instantiates a new cookie. The `RectCookieSpawner` places cookies within random bounds for the open field, whereas the `MazeCookieSpawner` identifies walkable coordinates within the maze layout using generated cell lists.

The maze was generated using an iterative random Depth-First Search (DFS) algorithm [12].

E. Display

The rendering of the interface is performed by the `GraphicsManager` using the Adafruit GFX-compatible `Arduino_GFX` library [13].

Transmitting a full 16-bit color (RGB565) frame (240×280 pixels) requires writing $131.25\,\mathrm{KB}$ of data. Since SPI transfers one bit per clock cycle, a $27\,\mathrm{MHz}$ bus provides a theoretical

throughput of 27 Mbit/s, resulting in a frame transmission time of approximately 40 ms, derived from the ratio between total frame size and bus bandwidth [14]. This exceeds the 20 ms budget of the 50 Hz game loop.

To circumvent this hardware bottleneck, the `GraphicsManager` implements a hybrid rendering architecture utilizing the dirty rectangles technique [15]:

- Full Redraw: Executed at the start of each round or when the game state changes. The manager clears the entire screen, draws the HUD, and renders the static maze layout. To optimize SPI bandwidth, horizontal spans of identical pixel states are drawn using a Run-Length Encoding (RLE) [16] approach instead of individual pixels.
- Partial Redraw: Used during active game updates. On every tick, the manager clears only the bounding box representing the ball's previous position. It recovers the background texture or any overlapping cookie graphics, then draws the ball at its new coordinate. This limits the active data transfer to approximately 1 KB per frame, keeping SPI communication time under 1 ms.

Volatile HUD elements, such as the score and elapsed time, are cached and updated only when their internal values change.

An example display frame of the labyrinth game is shown in Figure 5, while Figure 4 illustrates a loading screen for the game startup phase.

Fig. 4. Loading screen for the game startup phase on the ESP32-S3.

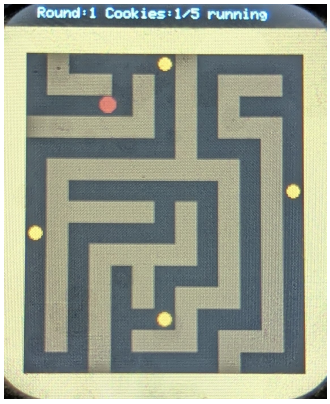

Fig. 5. Example display frame of the labyrinth game on the ESP32-S3.

F. MQTT Communication

The network layer connects to a HiveMQ Cloud broker over an encrypted TLS connection [1]. Network credentials and SSL certificates are compiled from the git-ignored `wifi_mqtt_secrets.h` file. Certificate validation is supported by synchronization with the NTP server pool via the `TimeManager` during initialization.

The device establishes two main communication paths:

- Command Topic (`game/command`): Receives control payloads from external dashboards, as illustrated in Figure 6. The incoming JSON content is processed by the static `CommandParser` class to adjust gameplay states or update physical and visual parameters on the fly [17]. A Universally Unique Identifier (UUID) [18] is used for tracking the origin of commands.
- Telemetry Topic (`game/telemetry`): Published telemetry data to dashboards at a 2 Hz rate, as shown in Figure 7. The `JsonBuilder` class structures the combined hardware information, game progress, configuration records, physics parameters, and raw sensor readings into a unified JSON format.

All communication packets default to Quality of Service 1 (QoS 1) with the retain flag set to false to avoid processing stale telemetry.

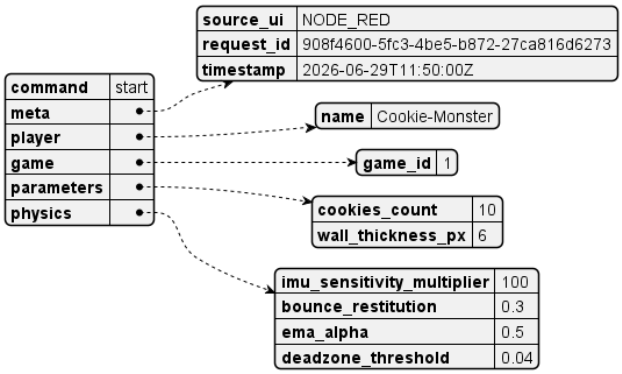

Fig. 6. Dashboard JSON Command Payload to the ESP32-S3.

IV. Android App

The BiteBound Android application acts as a remote controller and telemetry dashboard, facilitating secure, real-time bidirectional communication with the ESP32-S3 over MQTT. Developed in Kotlin [19] using the Model-View-ViewModel (MVVM) architecture [20], the system separates data and transmission processes from user interface rendering across four architectural layers:

- **Data Layer**: Manages profile persistence via Android SharedPreferences [21], telemetry deserialization, command serialization, and local data structures. This layer matches configuration constants, boundaries, and different ball presets to the ESP32 firmware specifications.
- **Network Layer**: Handles connectivity tasks using the HiveMQ client [22]. It coordinates SSL/TLS handshakes [23], topic subscriptions, automatic reconnection cycles, and message publication.
- **UI State and Logic Layer**: Orchestrated by a central ViewModel that maintains screen states, parses real-time telemetry updates, monitors connection heartbeats, and dispatches command requests.
- **UI Layer (Screens)**: Renders the interface using declarative, stateless composables via Jetpack Compose [24]. It features a configuration screen to initialize parameters (Figure 8) and an active gameplay dashboard that displays performance metrics as a radial progress bar [25], sensor diagnostics, and a dynamic canvas [26] tracking ball position and velocity (Figure 9).

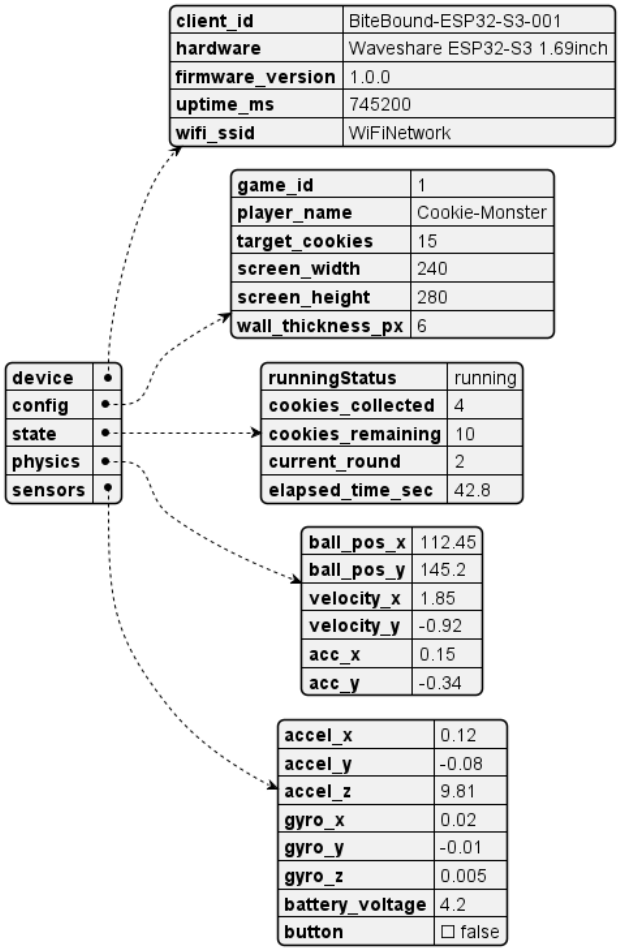

Fig. 7. ESP32-S3 JSON telemetry payload to the dashboards.

V. Node-RED

Node-RED [27] serves as a desktop-accessible companion to the Android application, providing alternative telemetry processing.

This system utilizes the code-first UIBUILDER package [28] rather than the officially deprecated dashboard [29] as the latter introduces potential security and support liabilities in modern Node.js runtimes. Additionally, UI Builder supports high-frequency telemetry streams by decoupling the client-side representation from the Node-RED backend using optimized WebSockets [30].

A. Workflow

The backend flow orchestrates MQTT setup, topic routing, debug logging, and state initialization. Incoming telemetry is pipelined directly into the `BiteBound Dashboard` uibuilder instance, while a parallel logging branch uses a filtering delay node to limit outputs. Manual command injections are available to bypass the UI for debugging purposes.

B. Dashboard & User Input

The uibuilder interface, as illustrated in Figure 10, offers a responsive, desktop-optimized web console that mirrors the style, capabilities and functionality of the Android app. The interface is modularized within the local web directory, separating markup, styling, and application logic:

- `index.html`: Establishes the DOM structure, dividing the workspace into cards.
- `index.css`: Implements a uniform, cookie-themed color palette using CSS variables to maintain styling coherence.
- `index.js`: Acts as the script entry point, initializing the client-side uibuilder library and listening for active WebSocket [31] updates from the flow.

The core logic processes telemetry inputs into standardized JavaScript objects, validates them against JSON specifications, and constructs device commands for WebSocket transmission. DOM rendering dynamically updates the HTML content based on the current state, though no data persistence is implemented.

VI. Project Management

The BiteBound project was executed by a two-person team, with each member contributing to all aspects of the project.

A. Project Planning

The development of BiteBound followed an incremental path, prioritizing the definition of interface contracts and network infrastructure before implementing complex local logic such as the physics simulation and graphics rendering. The project was structured into four distinct milestones to manage dependencies and verify integration at each phase.

- **Milestone 1: Architecture and Basic Setup** This phase focused on creating the repository structure, planning extensive software architecture, establishing development environments, and configuring the basic ESP32-S3 firmware skeleton. Essential network modules (WiFi connectivity, secure MQTT communication via HiveMQ Cloud, NTP-based time synchronization) were implemented. Defining the standard JSON schemas for telemetry and command topics early in the timeline allowed the basic setups of the Android application and Node-RED dashboards to proceed in parallel with the firmware development.
- **Milestone 2: Hardware Implementation** During this milestone, the core offline mechanics of the ESP32 firmware were established. The 2D physics engine and cookie collection logic were designed first, followed by the integration of the Sensor Manager and maze generation. The graphics subsystem was then built, including an evaluation of display libraries. Finally, the firmware was fully orchestrated by the combination of the Game Engine for the game modes and the FreeRTOS-based dual-core task management.
- **Milestone 3: UI Enhancement and Refinement** This phase concentrated on system integration, user interface polishing, and debugging. The Android application and Node-RED dashboards were updated to support the finalized game orchestration logic, enabling dynamic

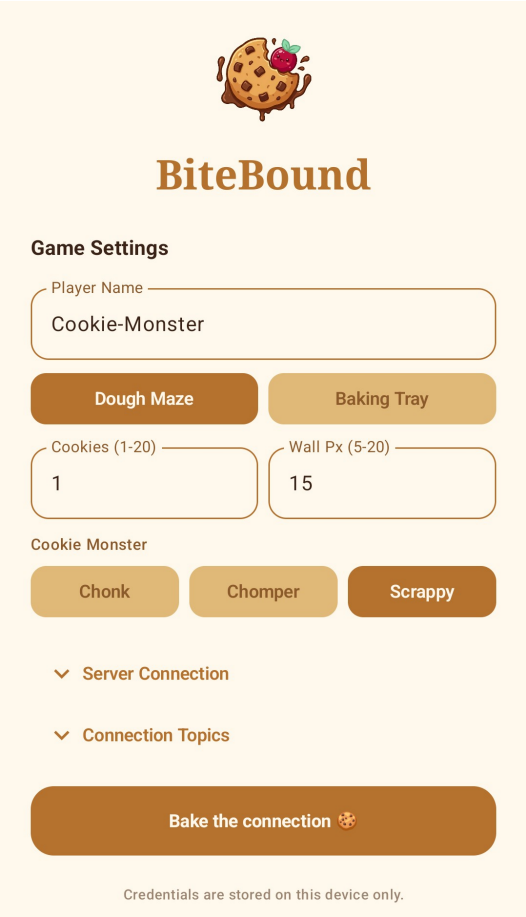

Fig. 8. Android app connection setup screen for MQTT broker and game configurations.

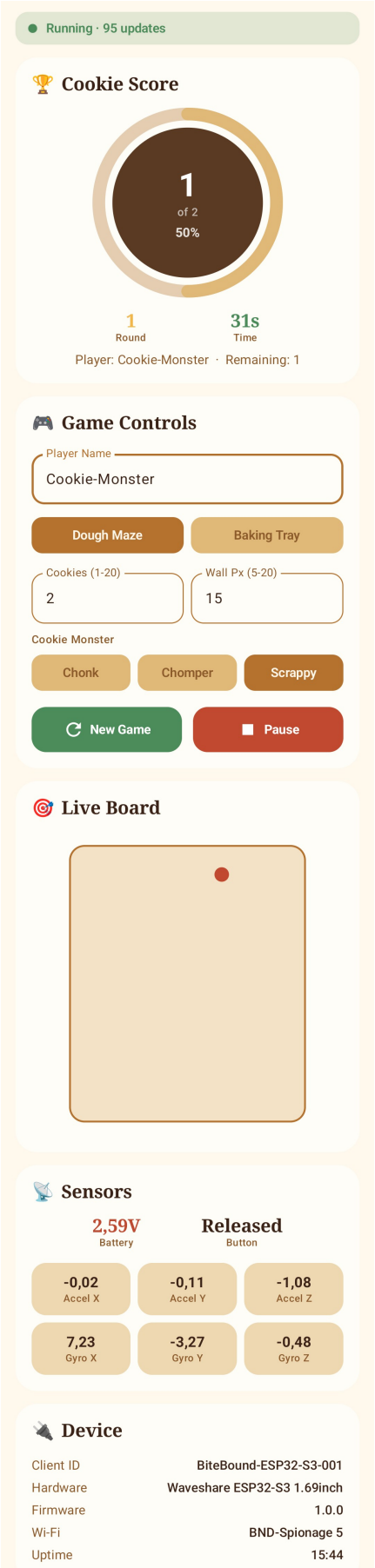

Fig. 9. Android app dashboard displaying real-time telemetry and controller interface.

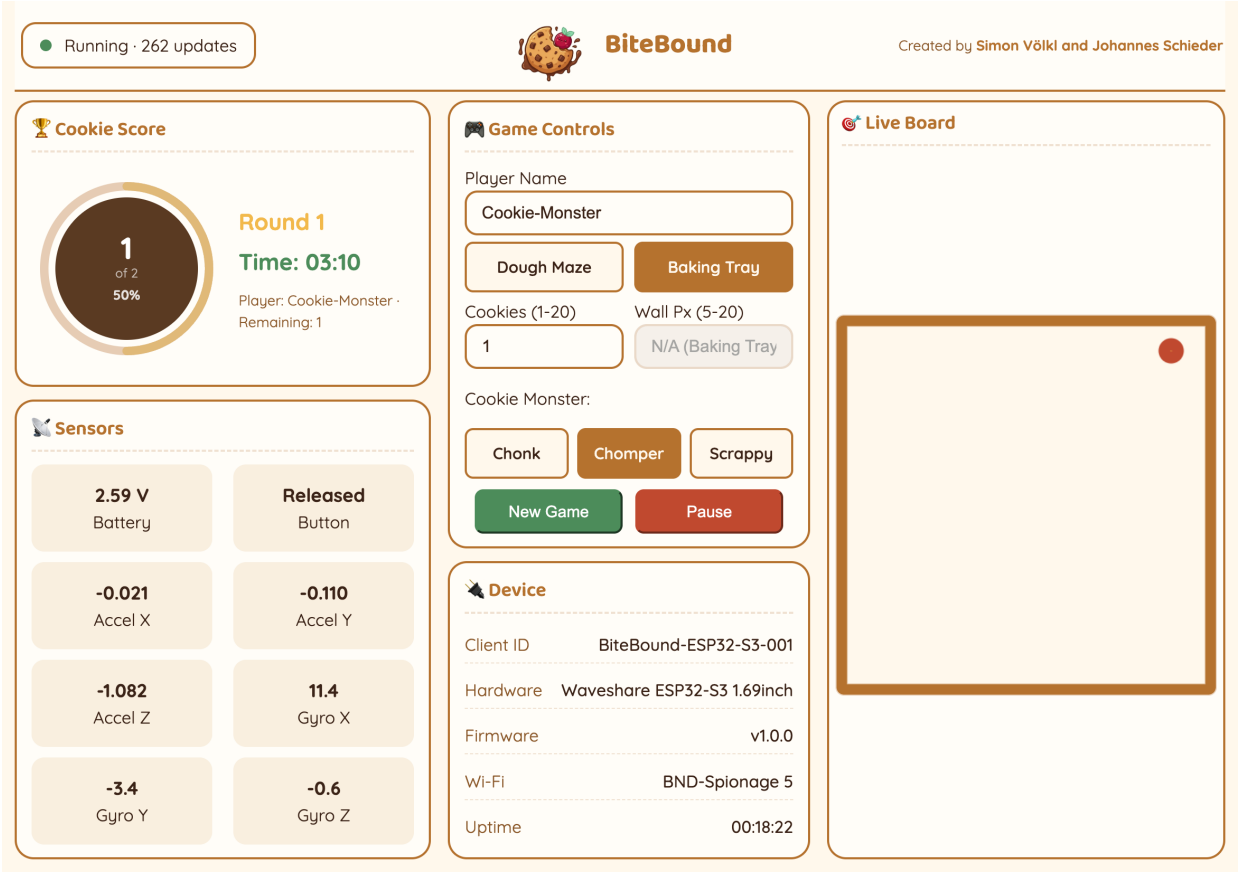

Fig. 10. Web-based operator dashboard rendered via uibuilder.

command and parameter handling. The project's visual identity with the logo, color scheme and assets was finalized, while last tests and bug-fixes were conducted to address firmware issues.

- **Milestone 4: Documentation** The final phase was dedicated to finalizing the technical report as documentation, enhancing the usage guide, and preparing the project for submission.

B. Task Dependencies

The decoupled software architecture introduced several critical dependencies: IMU calibration fed into physics input, JSON schema definitions enabled dashboard integration, and dual-core allocation required thread-safe state protection before merging the network and game codebases. These dependencies ensured that network managers and telemetry contracts were established first, physics and sensor processing were validated before graphics integration, and the multi-core architecture was introduced only after both stacks were independently stable.

C. Task Distribution

To coordinate development and ensure progress, tasks were tracked and managed using a GitHub Project board integrated directly with the code repository. This allowed issues to be linked to commits and Pull Requests (PRs).

To maintain code quality and structural integrity across the shared codebase, the development followed several guidelines:

- **Branch Protection:** Direct commits to the `main` branch were restricted. All feature updates and bug fixes were developed on separate branches.
- **Code Review:** Merging any branch into `main` required a Pull Request accompanied by the other team member's review and approval.
- **CI Pipeline**: A GitHub Actions pipeline was configured to automatically verify the compilation of the ESP32 firmware for both test and production builds.
- **Code Coverage Targets:** A unit test coverage of about 60% was targeted for the core ESP32 modules, while the Android application aimed for basic unit tests. Node-RED flows were excluded from automated testing due to their visual nature.
- **Docstrings:** All public methods and classes were documented with docstrings to enhance code readability and accessibility.

Transparency note: The unit test writing and docstring generation were partly assisted by AI.

VII. Discussion

During the development of BiteBound, several technical challenges were encountered, each requiring specific solutions to meet the set requirements.

As already mentioned in Section III-E, the SPI transmission overhead posed a significant challenge to maintaining a stable 50 Hz frame rate. By implementing a hybrid rendering system using the dirty rectangles technique, a substantial reduction in payload size was achieved, allowing for faster transmission times.

Recursive algorithmic approaches could lead to stack overflow errors without setting maze size limits. Therefore, the iterative DFS implementation for maze generation was a necessary design choice to generate arbitrary maze sizes without limiting the system to an upper bound.

Sensor value noise and drift were expected at the beginning of the project. Implementing a deadzone threshold and an EMA filter provided a straightforward solution to ensure stable physical interactions while still allowing for raw telemetry data to be available for diagnostic purposes.

Testing the physics engine after implementation revealed that at high tilt angles, the ball could tunnel through maze walls due to its accelerated velocity. The implementation of sub-stepping during collision detection effectively mitigated this issue, ensuring that boundaries were respected even at high velocities, as described in Section III-D.

During the architecture phase, the assumption was made that handling incoming dashboard commands and running the game loop on one core could cause significant frame drops. Early in development, the two-core possibility was explored, included in the initial design, and later implemented (see Section II). This decision proved crucial for the modular system design and, consequently, the testability, maintainability, and scalability of the system; the dual-core synchronization strategy worked as expected.

As only one LCD display was available, the other team member had to acquire a separate ESP32-S3 board to test and run the hardware code. Though, the other board only had one core, which limited the ability to test the dual-core synchronization strategy.

The HiveMQ broker was found to be inconsistent when handling multiple subscriptions simultaneously, leading to lags and disconnects. Still, no other broker was chosen for further development, as this issue only appeared sporadically.

VIII. Summary and Future Work

The BiteBound project demonstrates a practical approach to addressing the primary objectives established in Section I, specifically managing low-latency gameplay execution alongside secure, asynchronous network communication on resource-constrained hardware. By partitioning the timing-sensitive physics engine and rendering pipeline onto the Game Core and the communication onto the Network Core (Core 0), the system maintains a stable frame rate. This separation of concerns prevents network latency from interrupting physical gameplay while maintaining bidirectional synchronization with the Android and Node-RED companion interfaces, validating the multi-platform user experience proposed in the mission statement.

Future developments will focus on expanding the platform's physical interactivity, sensory feedback, and academic dissemination. To enrich the gameplay mechanics, upcoming iterations can incorporate onboard physical button inputs to trigger special, context-dependent actions during active play. Integrating a vibration motor or haptic sensor would further ground the physical interaction by providing physical feedback when the virtual ball collides with maze boundaries. Auditory feedback can also be expanded by implementing driver routines for the onboard buzzer, generating custom startup sequences, loading music, and distinctive alerts for coin-collection events. Finally, this technical report will be formatted into a scientific paper suitable for public submission to relevant mobile and ubiquitous computing venues, allowing for broader academic evaluation of the architecture.

References

[1] HiveMQ, "Hivemq," 2026. [Online]. Available: https://www.hivemq.com/

[2] MQTT.org, "Mqtt: The standard for iot messaging," 2026. [Online]. Available: https://mqtt.org/

[3] Arduino, "Arduino language reference," 2026. [Online]. Available: https://docs.arduino.cc/language-reference/

[4] Waveshare International Limited, "Esp32-s3 1.69inch touch display development board," 2026. [Online]. Available: https://docs.waveshare.com/ESP32-S3-Touch-LCD-1.69

[5] Waveshareteam, "Esp32-s3-touch-lcd-1.69 pin configuration," 2026. [Online]. Available: https://github.com/waveshareteam/ESP32-S3-Touch-LCD-1.69/blob/main/examples/Arduino/libraries/Mylibrary/pin_config.h

[6] Espressif Systems, "Esp-idf freertos api reference," 2026. [Online]. Available: https://docs.espressif.com/projects/esp-idf/en/v4.3/esp32/api-reference/system/freertos.html

[7] GeeksforGeeks, "Inter process communication (ipc)," 2026. [Online]. Available: https://www.geeksforgeeks.org/operating-systems/inter-process-communication-ipc/

[8] cppreference.com, "Resource acquisition is initialization (raii)," 2026. [Online]. Available: https://en.cppreference.com/w/cpp/language/raii

[9] F. Klinker, "Exponential moving average versus moving exponential average," *Mathematische Semesterberichte*, vol. 58, no. 1, pp. 97–107, 2011.

[10] O. Trnka, M. Hartman, and K. Svoboda, "An alternative semi-implicit euler method for the integration of highly stiff nonlinear differential equations," *Computers & Chemical Engineering*, vol. 21, no. 3, pp. 277–282, 1997. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0098135495002669

[11] J.-H. Wang, R. Setaluri, D. L. James, and D. K. Pai, "Bounce maps: an improved restitution model for real-time rigid-body impact." *ACM Trans. Graph.*, vol. 36, no. 4, pp. 150–162, 2017.

[12] NacerKroudir, "Randomized depth-first-search algorithm for maze generation," 2026. [Online]. Available: https://medium.com/@nacerkroudir/randomized-depth-first-search-algorithm-for-maze-generation-fb2d83702742

[13] Moon On Our Nation, "Arduino_gfx_library," 2026. [Online]. Available: https://github.com/moononournation/Arduino_GFX

[14] CompuTools, "Spi calculator," 2026. [Online]. Available: https://www.compu-tools.com/spi/

[15] StudyRaid, "Using dirty rect animation techniques," 2025. [Online]. Available: https://app.studyraid.com/en/read/15006/518776/using-dirty-rect-animation-techniques

[16] L. L. Peterson and B. S. Davie, *Computer Networks: A Systems Approach*, 6th ed. Morgan Kaufmann, 2023.

[17] JSON.org, "Introducing json," 2026. [Online]. Available: https://www.json.org/json-en.html

[18] A. Sharda, "Understanding how uuids are generated," 2020. [Online]. Available: https://digitalbunker.dev/understanding-how-uuids-are-generated/

[19] JetBrains, "Kotlin programming language," 2026. [Online]. Available: https://kotlinlang.org/

[20] Microsoft, "Model view viewmodel (mvvm)," 2026. [Online]. Available: https://learn.microsoft.com/de-de/dotnet/architecture/maui/mvvm

[21] Google, "Save simple data with sharedpreferences," 2026. [Online]. Available: https://developer.android.com/training/data-storage/shared-preferences

[22] HiveMQ, "Hivemq mqtt client," 2026. [Online]. Available: https://hivemq.github.io/hivemq-mqtt-client/

[23] Amazon Web Services, “The difference between ssl and tls,” 2026. [Online]. Available: https://aws.amazon.com/compare/the-difference-between-ssl-and-tls/
[24] Google, “Compose material 3,” 2026. [Online]. Available: https://developer.android.com/jetpack/androidx/releases/compose-material3
[25] D. Rupert, “Animated svg radial progress bars,” 2026. [Online]. Available: https://daverupert.com/2018/03/animated-svg-radial-progress-bars/
[26] W3Schools, “Html canvas graphics,” 2026. [Online]. Available: https://www.w3schools.com/html/html5_canvas.asp
[27] OpenJS Foundation, “Node-red – low-code programming for event-driven applications,” 2026. [Online]. Available: https://nodered.org
[28] J. Knight and Totally Information, “node-red-contrib-uibuilder.” [Online]. Available: https://github.com/TotallyInformation/node-red-contrib-uibuilder/
[29] Node-RED Dashboard Team, “Node-red dashboard (deprecated),” 2026. [Online]. Available: https://flows.nodered.org/node/node-red-dashboard
[30] Mozilla, “Websocket api,” 2026. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API
[31] J. Knight and Totally Information, “Uibuilderv7 for node-red,” 2026. [Online]. Available: https://totallyinformation.github.io/node-red-contrib-uibuilder/#/